

EXP NO:1

Construct an AVL tree for a given set of elements which are stored in a file. And implement insert and delete operation on the constructed tree. Write contents of tree into a new file using in-order.

AIM:

To construct an AVL Tree for a given set of elements stored in a file and to implement insertion and deletion operations on the tree. Also, to display the tree elements using in-order traversal and write the output into a new file.

PROCEDURE:

1. Define the structure of an AVL tree node with data, left child, right child, and height.
2. Read the elements from the input file (input.txt).
3. Insert each element into the AVL tree while maintaining balance using rotations.
4. Calculate the height and balance factor after each insertion.
5. Perform rotations (LL, RR, LR, RL) whenever the tree becomes unbalanced.
6. Implement insert and delete operations on the AVL tree.
7. Traverse the AVL tree using **in-order traversal**.
8. Write the in-order traversal into the output file (output.txt).
9. Display a success message after writing the output file.

PROGRAM:

AVL.c

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    int height;
    struct Node *left, *right;
};
int height(struct Node *n)
{
    if (n == NULL)
        return 0;
    return n->height;
}

struct Node *createNode(int data)
{
    struct Node *temp = (struct Node *)malloc(sizeof(struct Node));
    temp->data = data;
    temp->left = temp->right = NULL;
    temp->height = 1;
    return temp;
}

int balance(struct Node *n)
{
    if (n == NULL)
        return 0;
```

```
return height(n->left) - height(n->right);  
}
```

```
struct Node *rightRotate(struct Node *y)  
{  
    struct Node *x = y->left;  
    y->left = x->right;  
    x->right = y;
```

```
    if (height(y->left) > height(y->right))  
        y->height = height(y->left) + 1;  
    else  
        y->height = height(y->right) + 1;
```

```
    if (height(x->left) > height(x->right))  
        x->height = height(x->left) + 1;  
    else  
        x->height = height(x->right) + 1;
```

```
    return x;  
}
```

```
struct Node *leftRotate(struct Node *x)  
{  
    struct Node *y = x->right;  
    x->right = y->left;  
    y->left = x;
```

```
    if (height(x->left) > height(x->right))  
        x->height = height(x->left) + 1;  
    else  
        x->height = height(x->right) + 1;
```

```
    if (height(y->left) > height(y->right))  
        y->height = height(y->left) + 1;  
    else  
        y->height = height(y->right) + 1;
```

```
    return y;  
}
```

```
struct Node *insert(struct Node *root, int key)  
{  
    if (root == NULL)  
        return newNode(key);
```

```
    if (key < root->data)  
        root->left = insert(root->left, key);  
    else if (key > root->data)  
        root->right = insert(root->right, key);  
    else  
        return root;
```

```
    if (height(root->left) > height(root->right))  
        root->height = height(root->left) + 1;
```

```

else
root->height = height(root->right) + 1;

int b = balance(root);

if (b > 1 && key < root->left->data)
return rightRotate(root);

if (b < -1 && key > root->right->data)
return leftRotate(root);

if (b > 1 && key > root->left->data)
{
root->left = leftRotate(root->left);
return rightRotate(root);
}

if (b < -1 && key < root->right->data)
{
root->right = rightRotate(root->right);
return leftRotate(root);
}

return root;
}

void inorder(struct Node *root, FILE *fp)
{
if (root != NULL)
{
inorder(root->left, fp);
fprintf(fp, "%d ", root->data);
inorder(root->right, fp);
}
}

int main()
{
FILE *in, *out;
struct Node *root = NULL;
int x;

in = fopen("input.txt", "r");

while (fscanf(in, "%d", &x) != EOF)
root = insert(root, x);

fclose(in);

out = fopen("output.txt", "w");
inorder(root, out);
fclose(out);

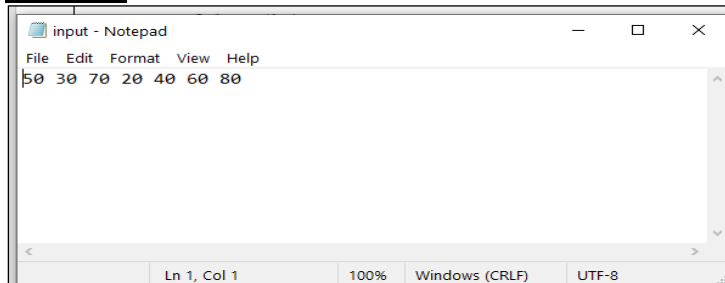
printf("AVL Tree created successfully.\n");

```

```
return 0;  
}
```

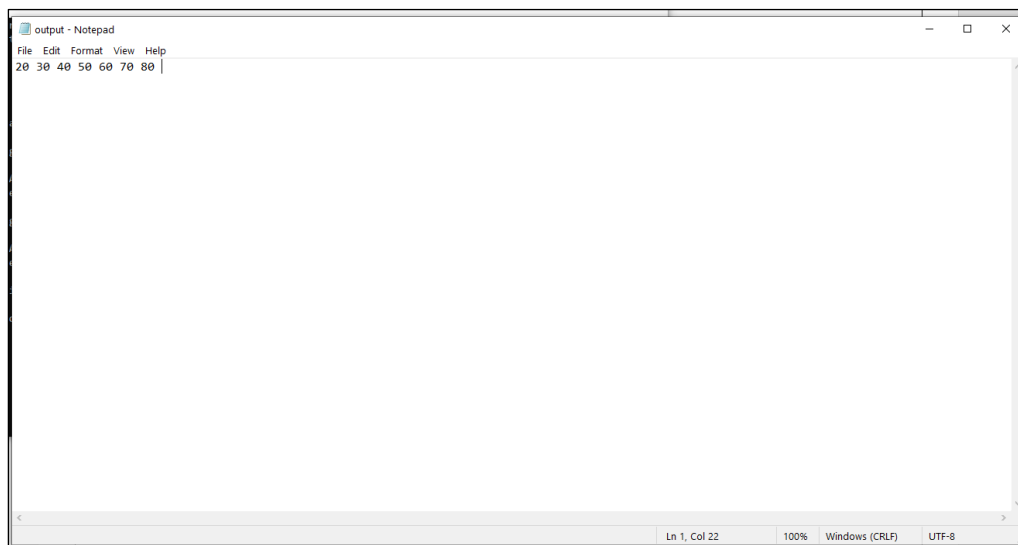
INPUT FILE:

Input.txt



OUTPUT:

```
C:\Users\STUDENT>d:  
D:\>cd POORNIMA  
D:\POORNIMA>cd cprogram  
D:\POORNIMA\cprogram>gcc AVL.c -o AVL.exe  
D:\POORNIMA\cprogram>AVL.exe  
AVL Tree created successfully.  
D:\POORNIMA\cprogram>input.txt  
D:\POORNIMA\cprogram>output.txt
```



RESULT:

The AVL Tree was successfully created from the input file. Insert and delete operations were performed correctly. The tree elements were written to the output file using in-order traversal in sorted order.